

[Algorithm]

그래프 탐색 알고리즘

인하대학교 소프트웨어융합공학과

김종훈 교수



❖ 목차

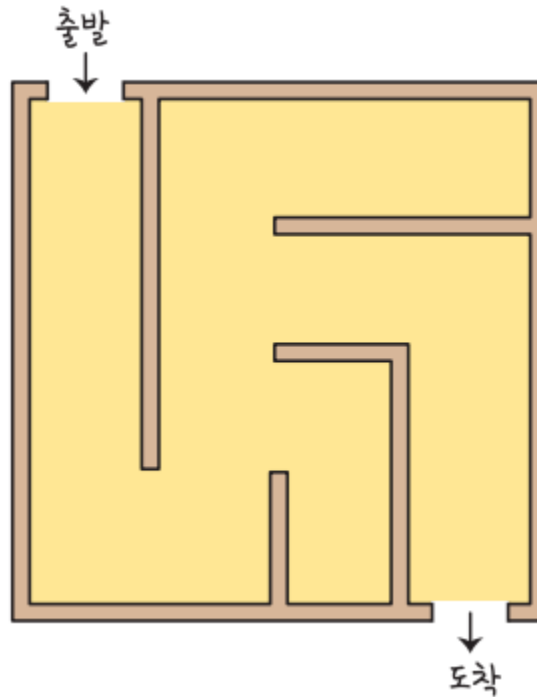
- 1. 깊이 우선 탐색
- 2. 너비 우선 탐색
- 3. 그래프의 구현
- 4. 알고리즘의 구현
- 5. 예제



1. 깊이 우선 탐색

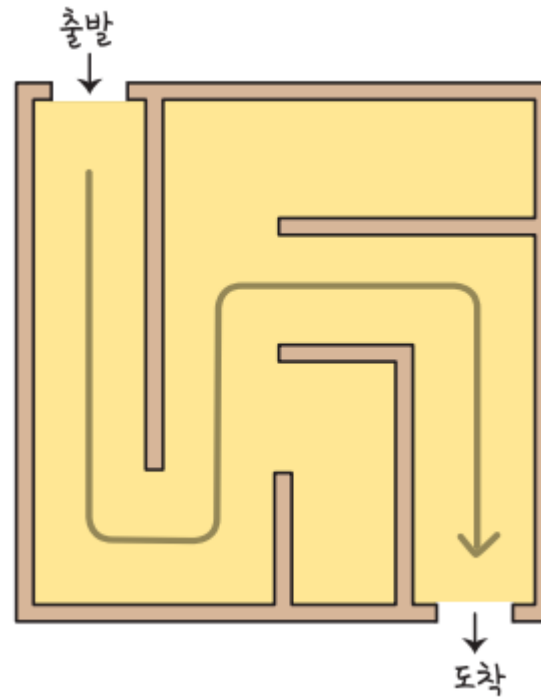
❖ 미로 찾기 알고리즘 문제

- 다음 그림과 같이 미로의 형태와 출발점과 도착점이 주어졌을 때
- 출발점에서 도착점까지 가기 위한 최단 경로를 찾는 알고리즘을 만들어 보세요.



1. 깊이 우선 탐색

- ## ■ 주어진 미로는 굉장히 간단한 문제



연필을 들면 바로 해결할 수 있음



1. 깊이 우선 탐색

❖ 문제 분석과 모델링

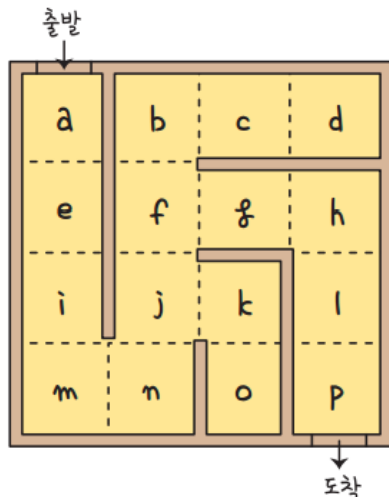
- 이 문제를 컴퓨터에게 풀어 보라고 하려면 어떻게 해야 할까?
- 사람에게는 쉽지만 컴퓨터에게 이 문제를 이해하고 풀게 하긴 어려움
- 이때 필요한 것이 바로 '모델링(모형화)'
 - 모델링이란 주어진 현실의 문제를 정형화하거나 단순화하여 수학이나 컴퓨터 프로그램으로 쉽게 설명할 수 있도록 다시 표현하는 것
 - 모델링은 자연이나 사회현상을 사람의 언어로 표현한 문제를 컴퓨터가 쉽게 이해할 수 있도록 수학적식이나 프로그래밍 언어로 번역하는 절차



1. 깊이 우선 탐색

❖ 문제 분석과 모델링

- 그래프를 이용해 미로 찾기 문제를 단계별로 모델링해 보자
- 일단 미로를 풀려면 미로 안의 공간을 정형화해야 함
- 퍼즐은 4×4 로 구성된 간단한 미로
- 이동 가능한 위치를 각각의 구역으로 나누고, 구역마다 알파벳으로 이름을 붙이면 아래와 같음



미로 안의 공간을 정형화한 결과

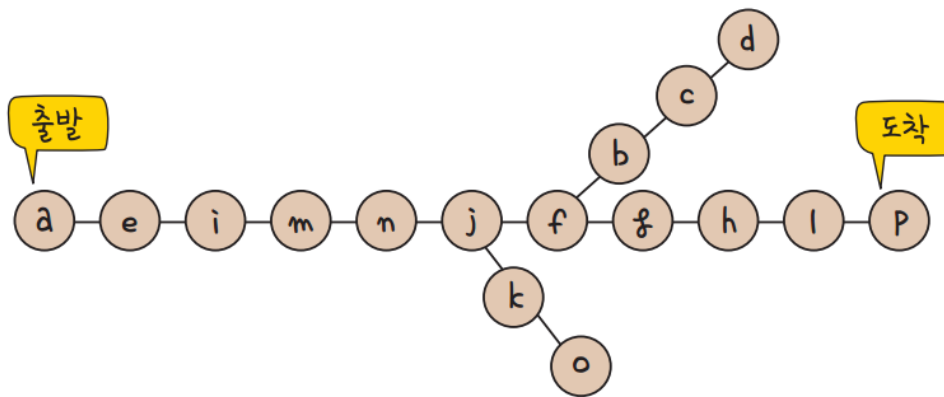
출발점 a에서 시작하여 벽으로 막히지 않은 위치로 차례로 이동하여 도착점 p에 이르는 가장 짧은 경로를 구하고, 그 과정에서 지나간 위치의 이름을 출력해 보세요.



1. 깊이 우선 탐색

❖ 문제 분석과 모델링

- 최종 결과를 얻으려면 각 위치 사이의 관계를 컴퓨터에게 알려 줘야 하고 실제로 미로를 푸는 알고리즘도 만들어야 함
- 이 문제는 그래프 탐색 문제와 같음
- 위치 열여섯 개를 각각 꼭지점으로 만들고, 각 위치에서 벽으로 막히지 않아 이동할 수 있는 이웃한 위치를 모두 선으로 연결



주어진 미로를 그래프로 표현

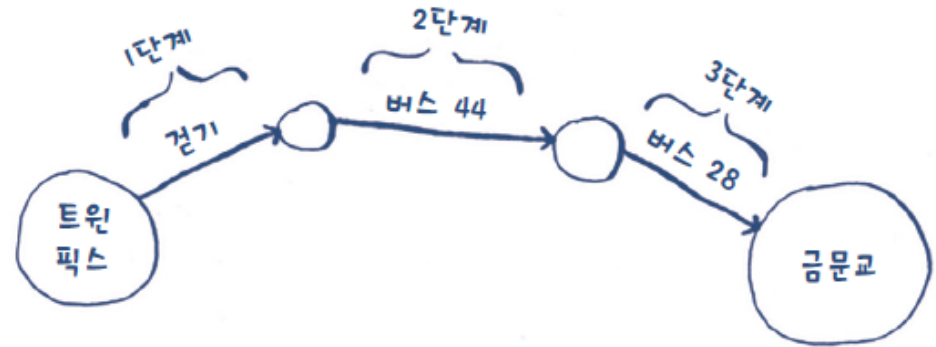


2. 너비 우선 탐색

❖ 최단 경로 문제 (Shortest-path Problem)

■ Ex) 도시의 한 지점에서 다른 지점으로 가는 최단 경로

- 문제를 그래프로 모형화
- 너비 우선 탐색으로 문제 해결



❖ 너비 우선 탐색 (BFS, Breadth-First Search) 알고리즘

❖ 너비 우선 탐색 활용하여 두 항목 간 최단 경로 찾기

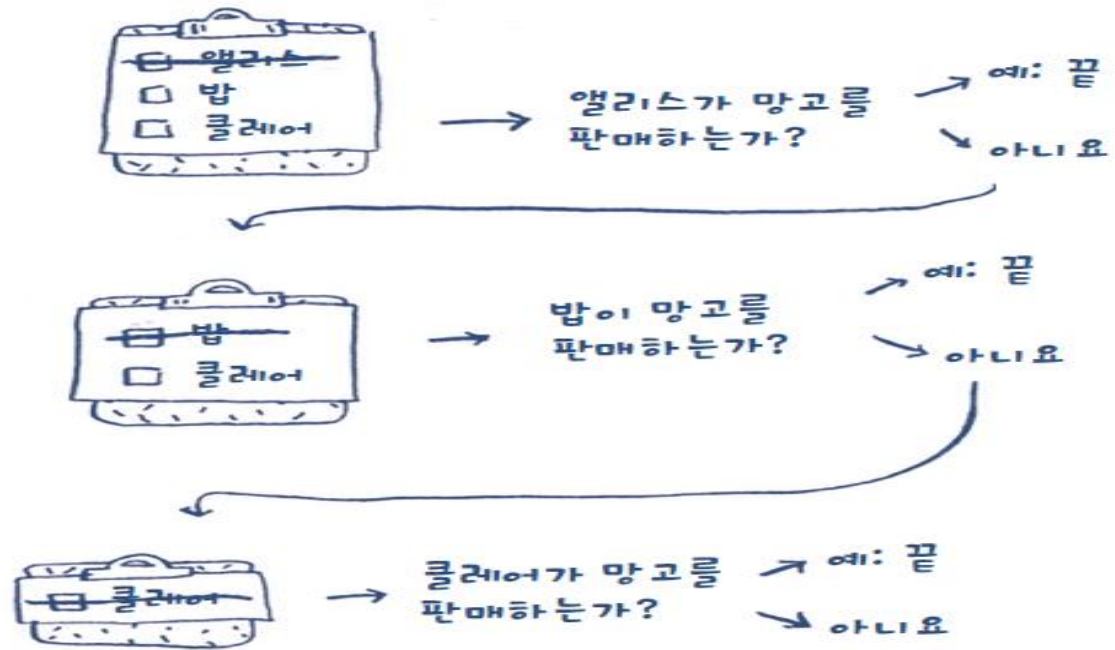
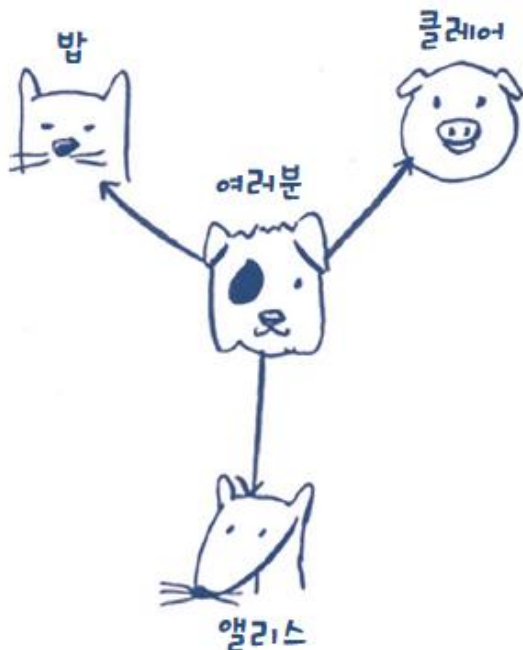
- 가장 가까운 의사를 찾으려면?



2. 너비 우선 탐색

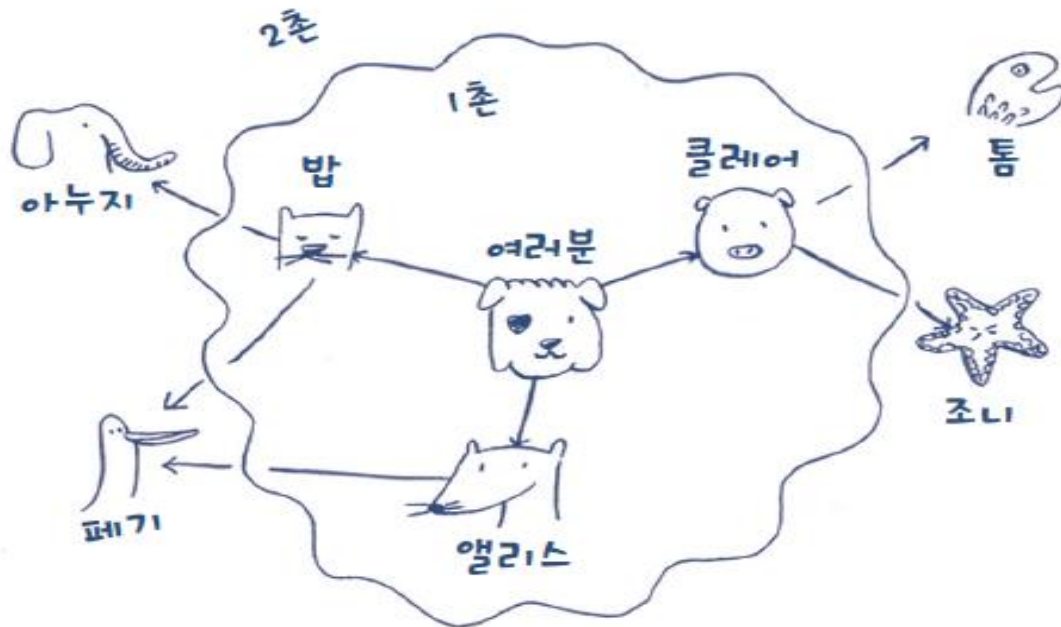
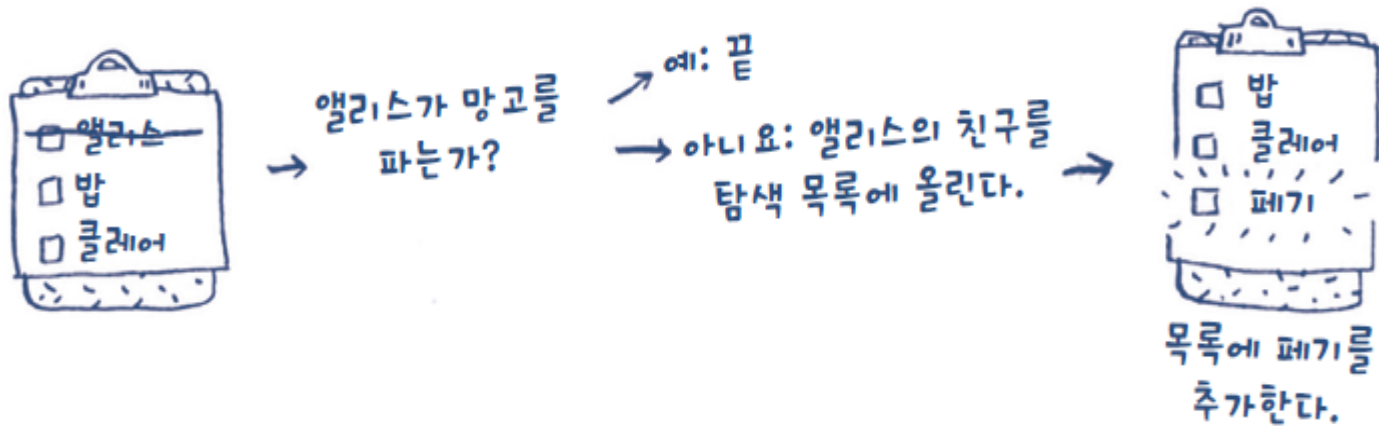
❖ 최단 경로 찾기

- 정점 A에서 정점 B로 가는 경로가 존재하는가?
- 정점 A에서 정점 B로 가는 최단 경로는 무엇인가?
- 필요한 요소에 도달할 때까지 전체 네트워크 탐색
- Ex) 인적 네트워크에서 필요한 직업의 인물(과일 판매상) 찾기



2. 너비 우선 탐색

- 직접적인 친구들 중에 없는 경우 친구의 친구 검색



2. 너비 우선 탐색

- 너비 우선 탐색이 진행될수록 탐색 범위는 출발점에서 멀어짐
 - 목록에 더해진 순서(1촌, 2촌, ...)대로 탐색

❖ 큐 (Queue)

■ 삽입 (enqueue)

- 큐에 항목을 더함

■ 제거 (dequeue)

- 큐에서 항목을 꺼냄
- 두 개 항목을 삽입하면 첫 번째로 삽입된 항목이 먼저 제거

■ 선입선출 자료구조 (FIFO, First In First Out)

- 큐 안의 원소에 임의로 접근할 수 없음
- 큐 사용하여 목록에 먼저 추가된 사람들을 먼저 꺼내서 탐색



3. 그래프의 구현

❖ 해시 테이블 사용

- 한 정점에 이웃하는 정점을 할당

```
graph = {}
```

```
graph["you"] = ["alice", "bob", "claire"]
```

- graph["you"]
 - 모든 이웃

```
graph = {}
```

```
graph["you"] = ["alice", "bob", "claire"]
```

```
graph["bob"] = ["anuj", "peggy"]
```

```
graph["alice"] = ["peggy"]
```

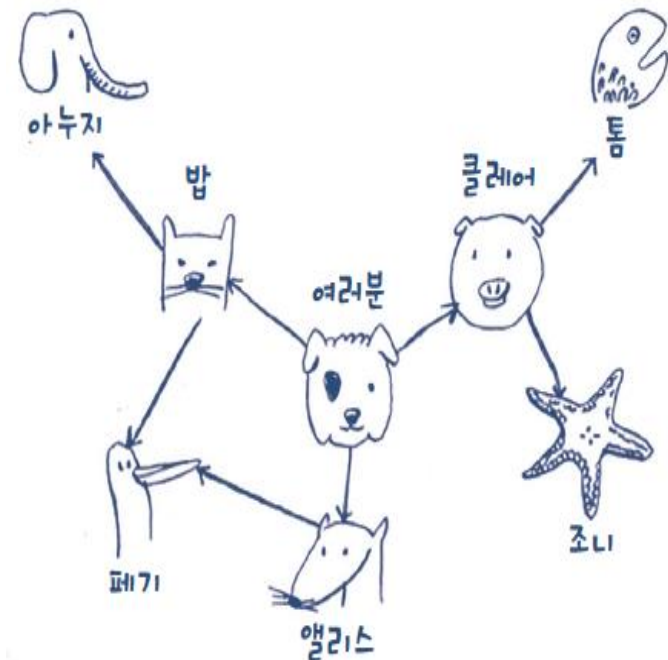
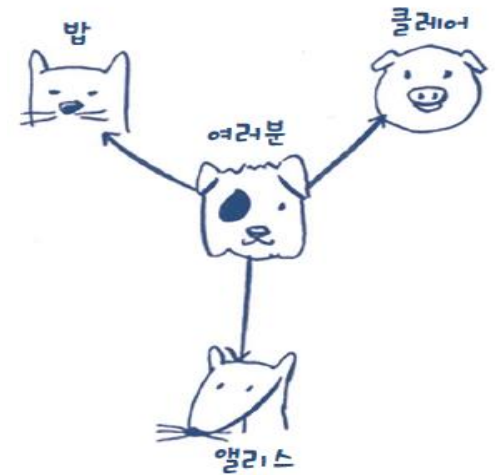
```
graph["claire"] = ["thom", "jonny"]
```

```
graph["anuj"] = []
```

```
graph["peggy"] = []
```

```
graph["thom"] = []
```

```
graph["jonny"] = []
```



4. 알고리즘의 구현

❖ 알고리즘이 구현되는 방식

- 큐를 생성
- 모든 이웃 목록을 탐색 큐에 추가

```
from collections import deque
```

```
search_queue = deque()
```

새 큐를 생성

```
search_queue += graph["you"]
```

모든 이웃을 탐색 큐에 추가

```
while search_queue:
```

큐가 비어 있지 않는 한 계속 실행

```
    person = search_queue.popleft()
```

큐의 첫 번째 사람을 꺼냄

```
    if person_is_seller(person):
```

망고 판매상인지 확인

```
        print person + " is a mango seller!"
```

망고 판매상이 맞음

```
        return True
```

```
    else:
```

```
        search_queue += graph[person]
```

망고 판매상이 아님. 모든
이웃을 탐색 목록에 추가

```
return False
```

여기에 도달했다는 것은 망고
판매상이 아무도 없다는 의미



4. 알고리즘의 구현

- 망고 판매상인지 확인하는 함수

```
def person_is_seller(name):  
    return name[-1] == 'm'
```

- 알고리즘은 다음의 경우에 종료
 - 망고 판매상을 발견하는 경우
 - 큐가 비는 경우 (망고상이 존재하지 않음)
- 어느 사람 확인 후에는 그 사람이 다시 탐색되지 않도록 해야
 - 무한 반복 가능

❖ 실행 시간

- $O(V+E)$
 - $O(\text{사람의 수} + \text{간선의 수})$
- 탐색 실행 최소시간: $O(\text{간선의 개수})$
- 큐에 모든 사람 추가하는 시간: $O(\text{사람의 수})$



4. 알고리즘의 구현



... 계속 ...



4. 알고리즘의 구현

■ 위 조건 고려한 최종 형태

```
def search(name):  
    search_queue = deque()  
    search_queue += graph[name]  
    searched = []  •———— 이 배열은 이미 확인한 사람을 추적하기 위한 것  
    while search_queue:  
        person = search_queue.popleft()  
        if not person in searched:  •———— 이전에 확인하지 않은 사람만 확인  
            if person_is_seller(person):  
                print person + " is a mango seller!"  
                return True  
            else:  
                search_queue += graph[person]  
                searched.append(person)  •———— 이 사람을 확인한 것으로 표시  
    return False  
  
search("you")
```



5. 예제

■ 인맥 찾기

```
def print_all_friends(g, start):
    qu = []      # 기억 장소1: 앞으로 처리해야 할 사람들을 큐에 저장
    done = set() # 기억 장소2: 이미 큐에 추가한 사람들을 집합에 기록(중복 방지)

    qu.append(start) # 자신을 큐에 넣고 시작
    done.add(start)  # 집합에도 추가

    while qu:        # 큐에 처리할 사람이 남아 있는 동안
        p = qu.pop(0) # 큐에서 처리 대상을 한 명 꺼내
        print(p)      # 이름을 출력하고
        for x in g[p]: # 그의 친구들 중에
            if x not in done: # 아직 큐에 추가된 적이 없는 사람을
                qu.append(x)  # 큐에 추가하고
                done.add(x)   # 집합에도 추가

# 친구 관계 리스트
# A와 B가 친구이면
# A의 친구 리스트에도 B가 나오고, B의 친구 리스트에도 A가 나옴
fr_info = {
    'Summer': ['John', 'Justin', 'Mike'],
    'John': ['Summer', 'Justin'],
    'Justin': ['John', 'Summer', 'Mike', 'May'],
    'Mike': ['Summer', 'Justin'],
    'May': ['Justin', 'Kim'],
    'Kim': ['May'],
    'Tom': ['Jerry'],
    'Jerry': ['Tom']
}

print_all_friends(fr_info, 'Summer')
print()
print_all_friends(fr_info, 'Jerry')
```





Thank You